Practical : Implementation of Automata and Formal Languages Using JFLAP 7.1

Aim : To implement and test different automata, grammars, formal-language concepts using JFLAP 7.1.

Software Requirements :

- JFLAP 7.1
- Java Runtime Environment / JDK

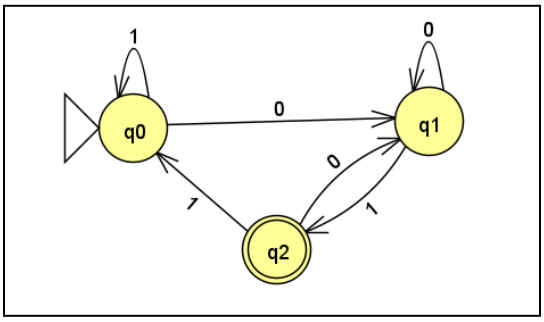
Questions and Implementation :

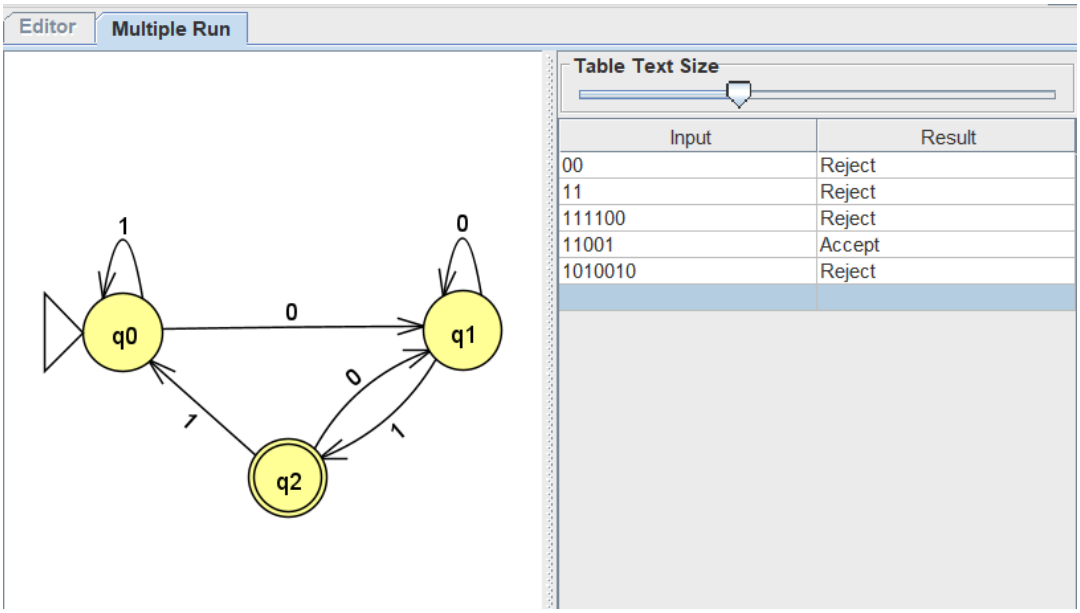
Question 1: Design a Deterministic Finite Automaton (DFA)

Aim: To design a DFA that accepts binary strings ending with 01.

Procedure:

1. Open JFLAP and select Finite Automaton.
2. Create the states and transitions required to recognize strings ending with 01.
3. Mark the initial and final states.
4. Test sample strings and observe acceptance or rejection.





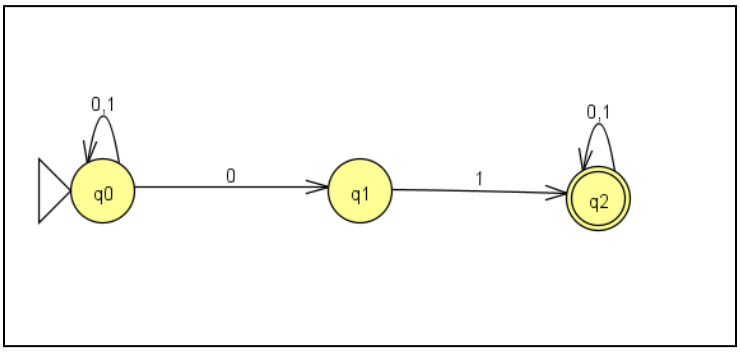
Result: The DFA was designed to accept binary strings ending with 01.

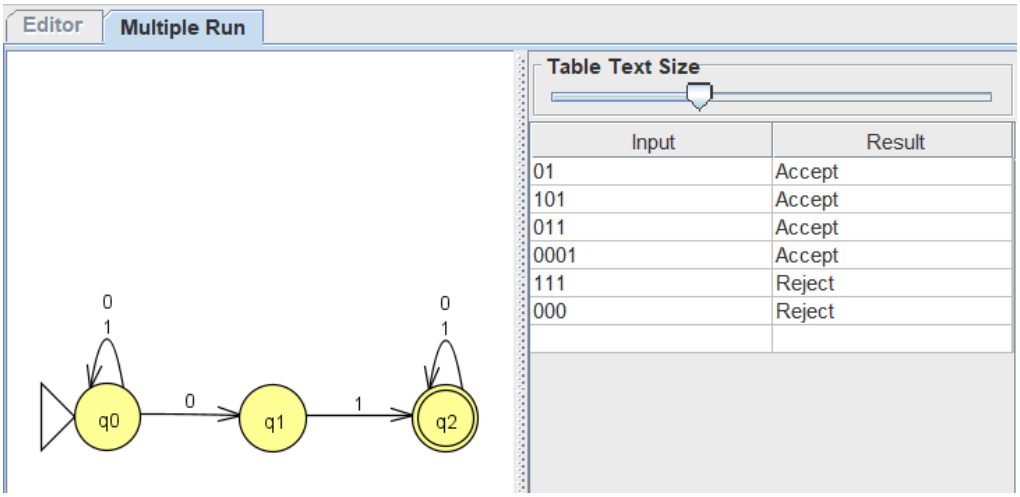
Question 2: Design a Nondeterministic Finite Automaton (NFA)

Aim: To design an NFA that accepts strings containing 01 as a substring.

Procedure:

1. Create the NFA in JFLAP.
2. Add the required states, transitions, and self-loops.
3. Mark the initial and final states.
4. Test strings containing and not containing 01.





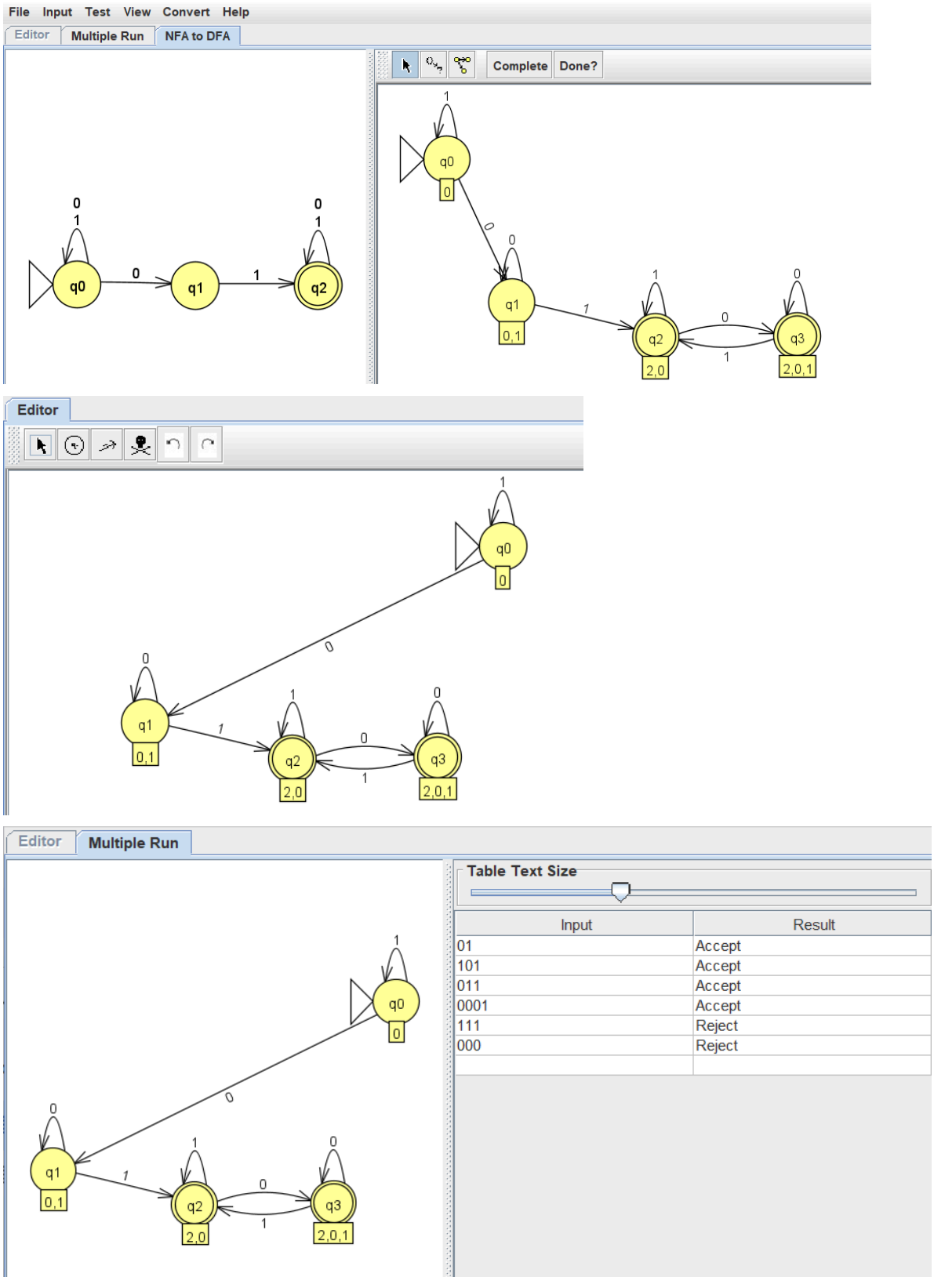
Result: The NFA was designed to accept strings containing 01.

Question 3: Convert an NFA into an Equivalent DFA

Aim: To convert an NFA into an equivalent DFA using JFLAP.

Procedure:

1. Open the NFA created for Question 2.
2. Select the NFA-to-DFA conversion option.
3. Inspect the generated DFA.
4. Test the same strings on both automata and compare the results.



Result: The NFA was converted into an equivalent DFA.

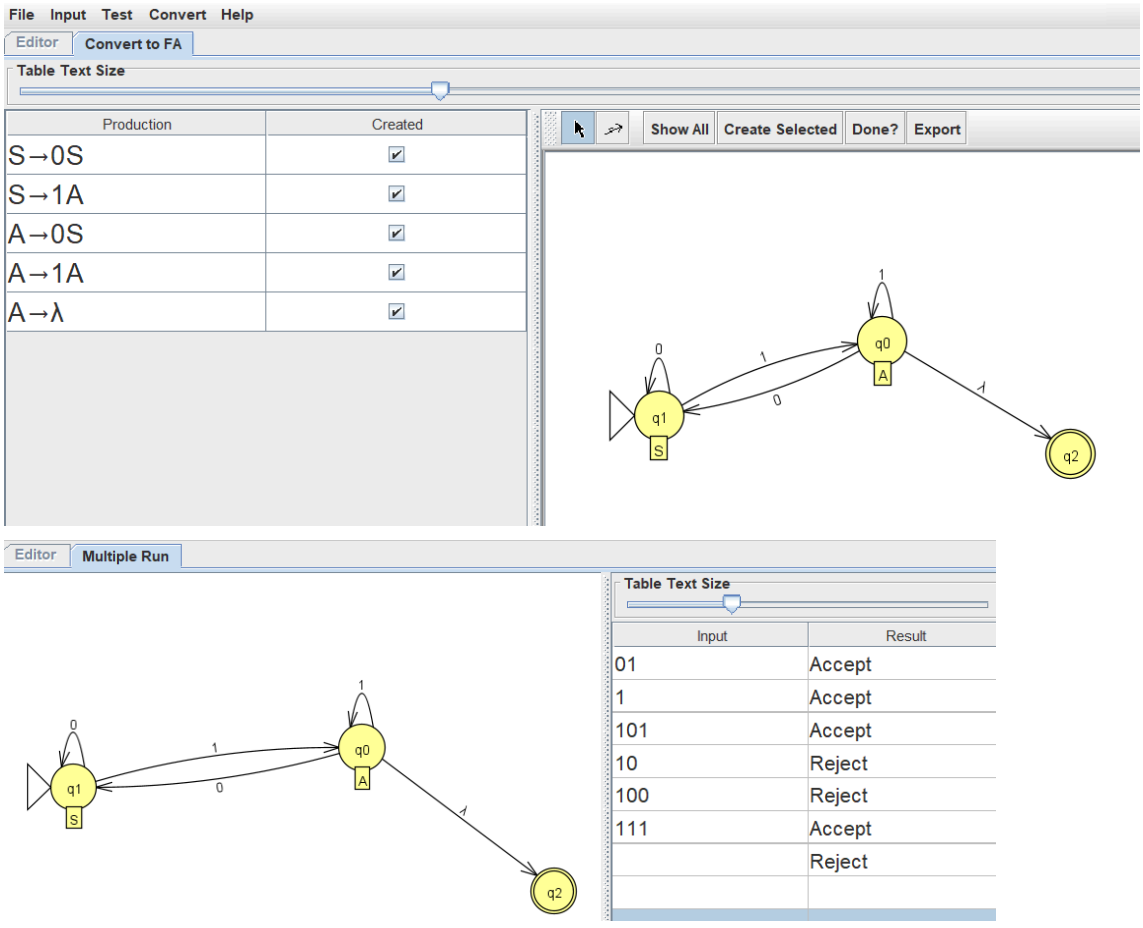
Question 4: Convert a Regular Grammar into a Finite Automaton

Aim: To convert a right-linear regular grammar into an equivalent finite automaton.

Grammar:
S → 0S | 1A
A → 0S | 1A | λ

Procedure:

1. Open the Grammar editor in JFLAP.
2. Enter the productions.
3. Convert the grammar into a finite automaton.
4. Test strings ending with 1 and strings not ending with 1.



Result: The regular grammar was converted into a finite automaton.

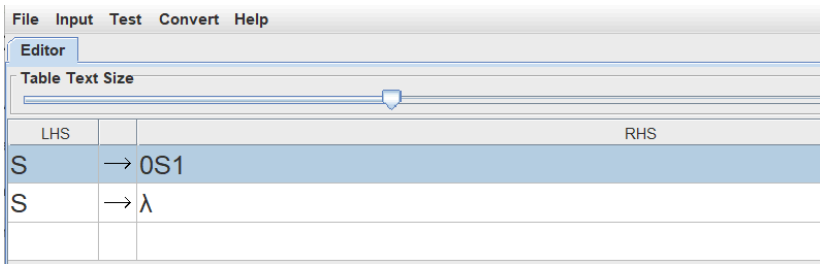
Question 5: Construct a Context-Free Grammar

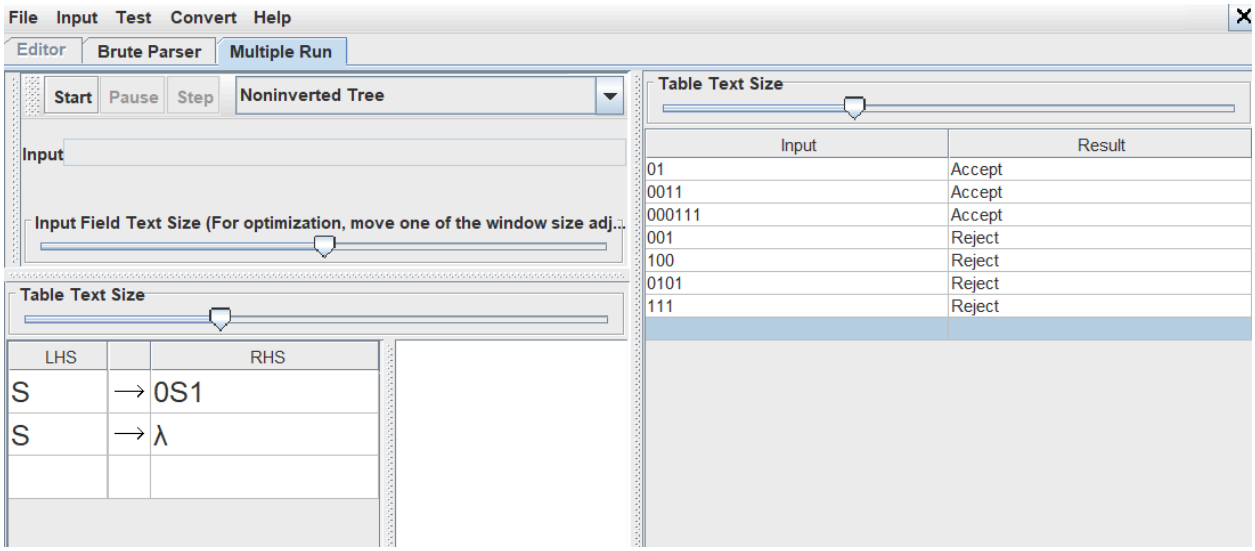
Aim: To construct a CFG for the language $L = \{0^n1^n \mid n \geq 0\}$.

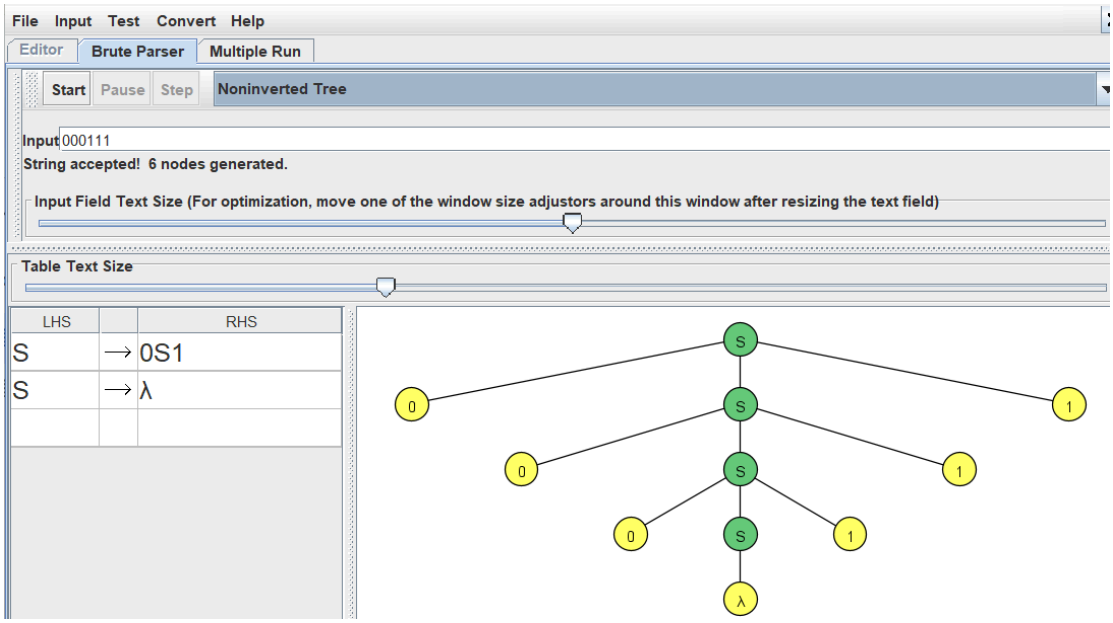
Grammar:
S → 0S1 | λ

Procedure:

1. Select Grammar in JFLAP.
2. Enter the production rules.
3. Generate strings using the grammar.
4. Test strings such as 01, 0011, and 000111.







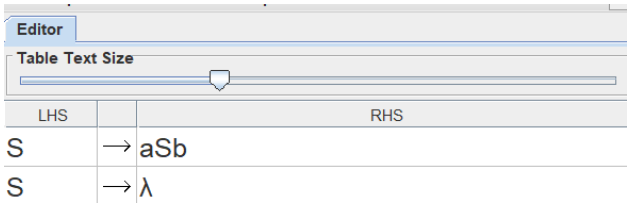
Result: The CFG generates strings with equal numbers of 0s followed by 1s.

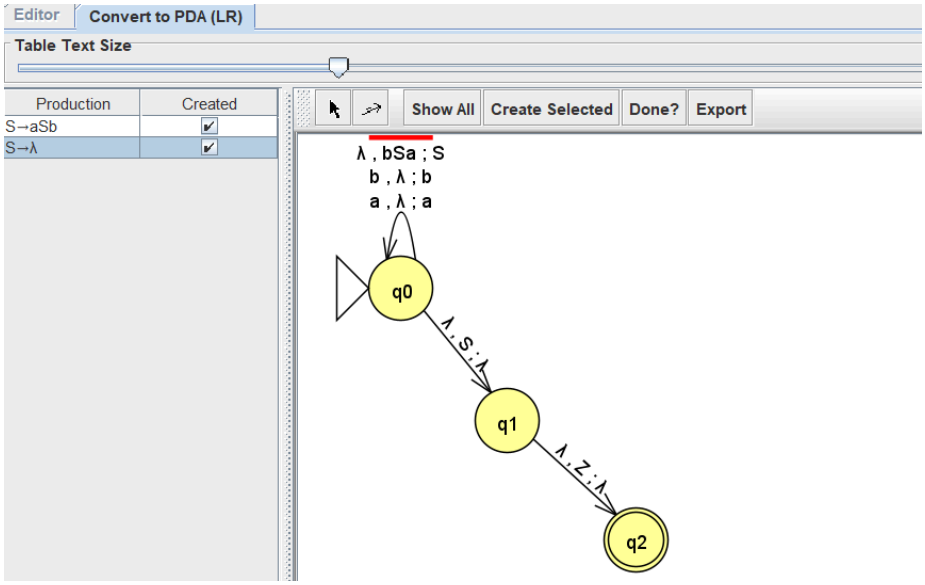
Question 6: LR Parsing

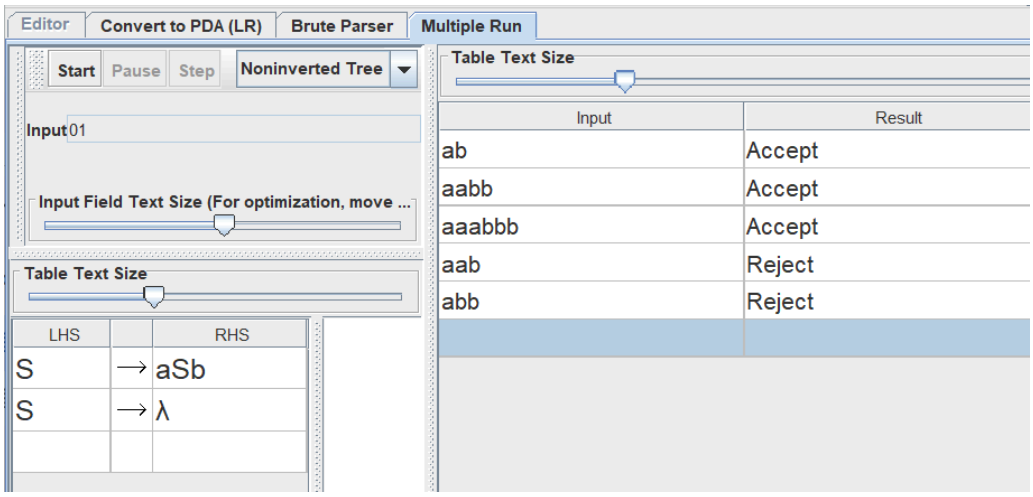
Aim: To explore LR parsing using the grammar and input tested in JFLAP.

Procedure:

1. Enter the grammar used in the completed exercise.
2. Open the appropriate LR parsing tool.
3. Inspect the LR items, states, transitions, or parsing table generated.
4. Test the input string and record the result.







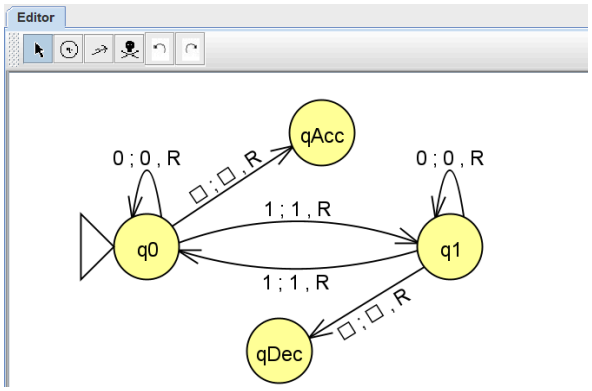
Result: The grammar and input were examined using the LR parsing facility in JFLAP.

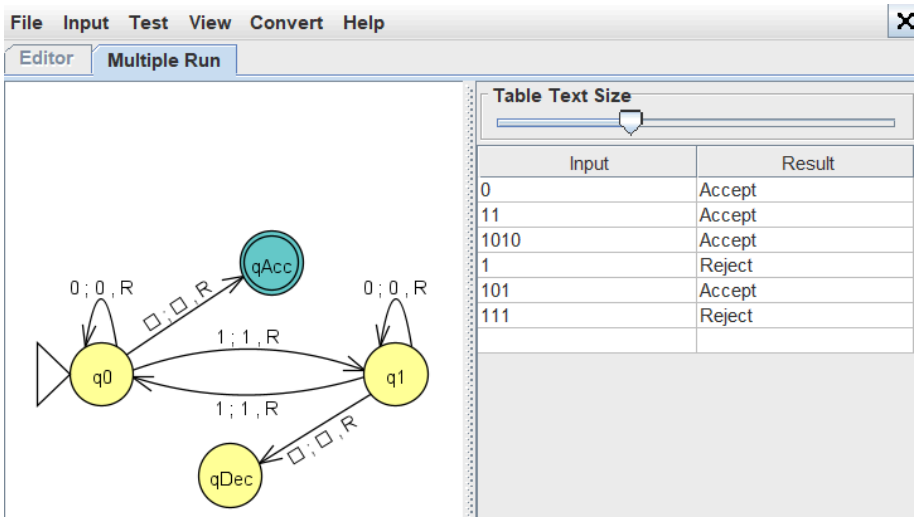
Question 7: Design a One-Tape Turing Machine

Aim: To design a one-tape Turing machine that accepts binary strings containing an even number of 1s.

Procedure:

1. Select Turing Machine in JFLAP.
2. Create states representing even and odd counts of 1s, along with accepting and rejecting
3. Add transitions for input symbols 0, 1, and the blank symbol.
4. Test strings containing even and odd numbers of 1s.





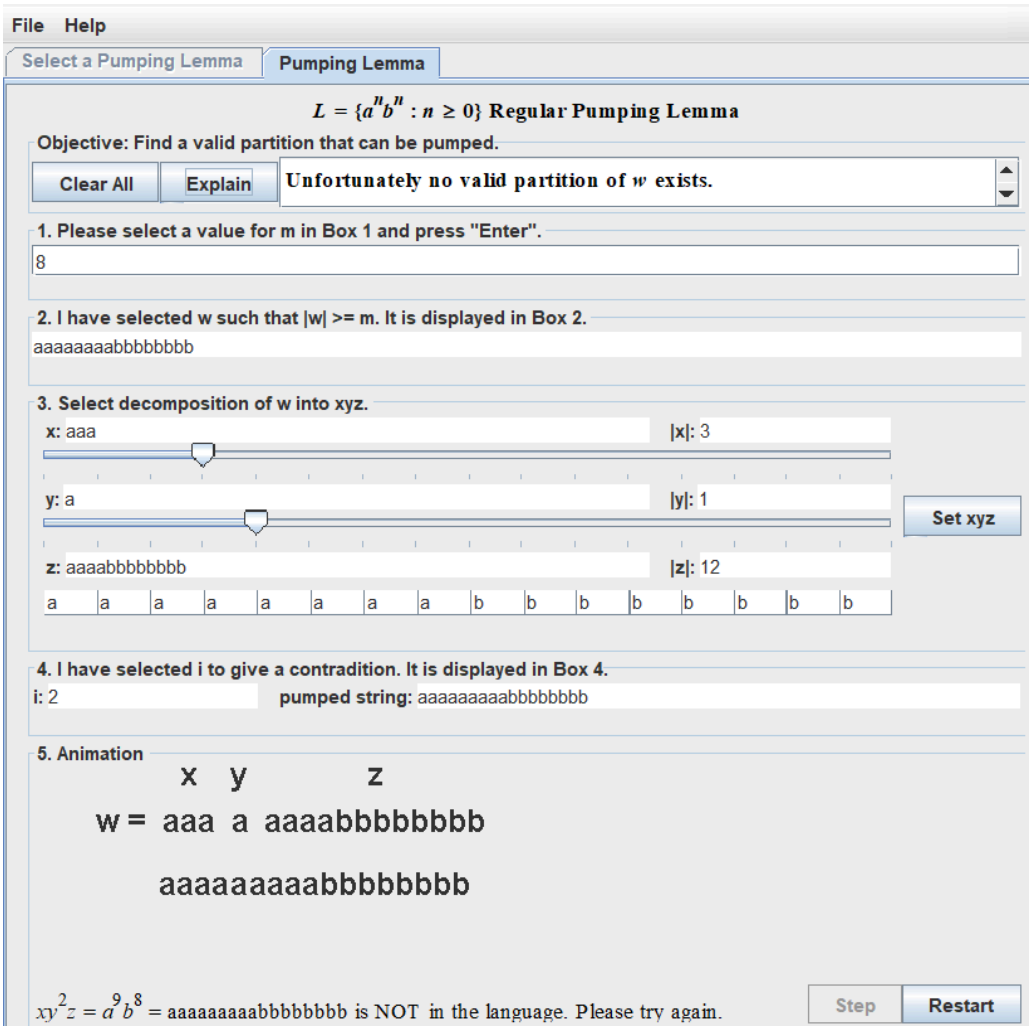
Result: The machine accepts strings containing an even number of 1s and rejects strings containing an odd number of 1s.

Question 8: Regular Pumping Lemma

Aim: To explore the regular pumping lemma using JFLAP and the language $L = \{a^n b^n \mid n \geq 0\}$.

Procedure:

1. Open the Regular Pumping Lemma exercise in JFLAP.
2. Select the language, if available, and enter the pumping length.
3. Follow the prompts to obtain a string and choose its decomposition into x, y, and z.
4. Use the Explain option to inspect the pumping result.



Result: The exercise demonstrates the pumping process for a non-regular language. The final result should be recorded according to JFLAP's actual output.

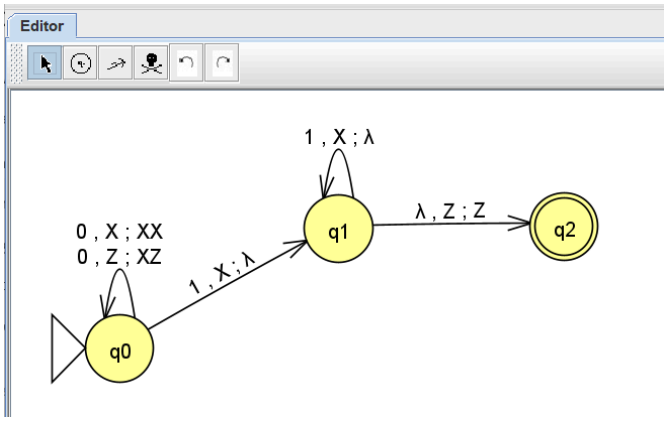
Question 9: Construct an NPDA for the language

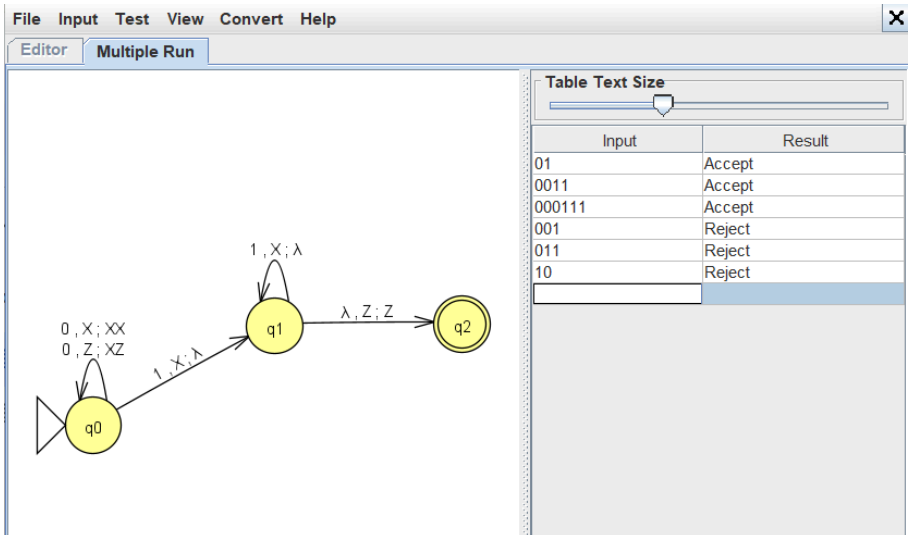
Aim : $L = \{0^n 1^n | n \geq 1\}$ and convert it into an equivalent Context-Free Grammar (CFG) using JFLAP 7.1.

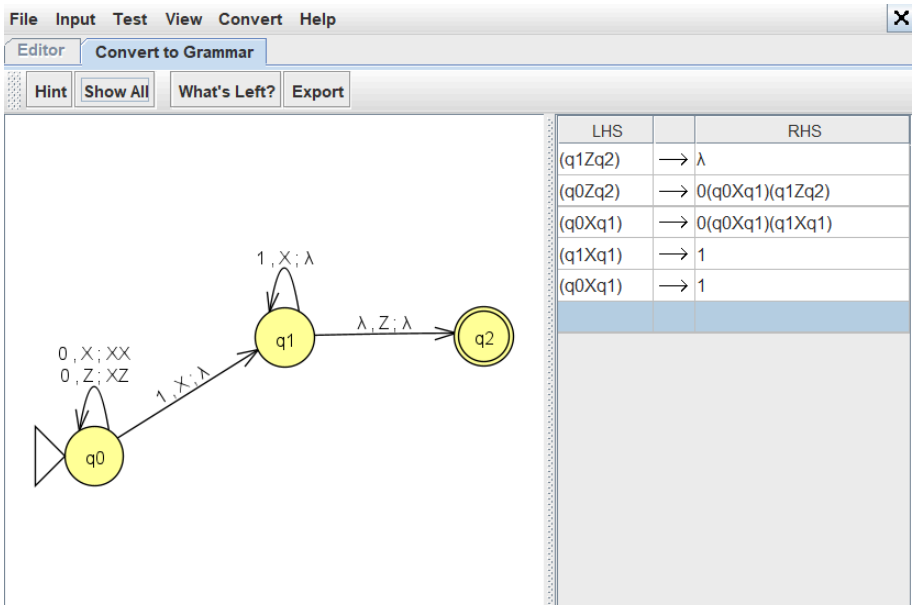
Procedure:

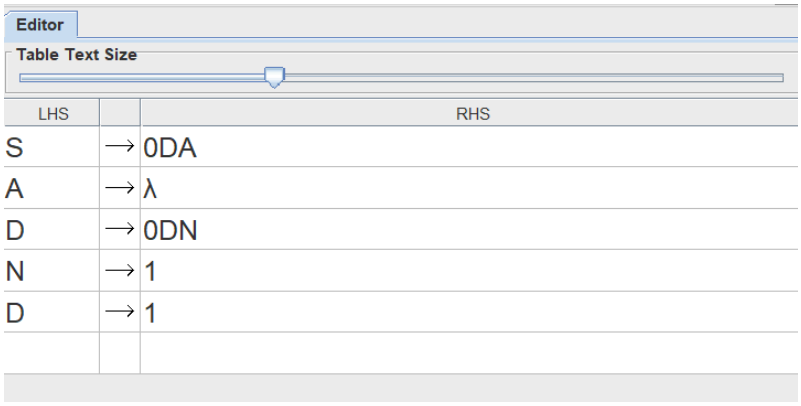
1. Open JFLAP 7.1 and select **Pushdown Automaton**.
2. Create three states: (q0), (q1), and (q2). Mark (q0) as the initial state and (q2) as the final state.
3. Define transitions to push a stack symbol (X) for each input `0` and pop one (X) for each input `1`.
4. Add a transition to reach the final state when all input symbols have been matched and the bottom-of-stack marker is reached.

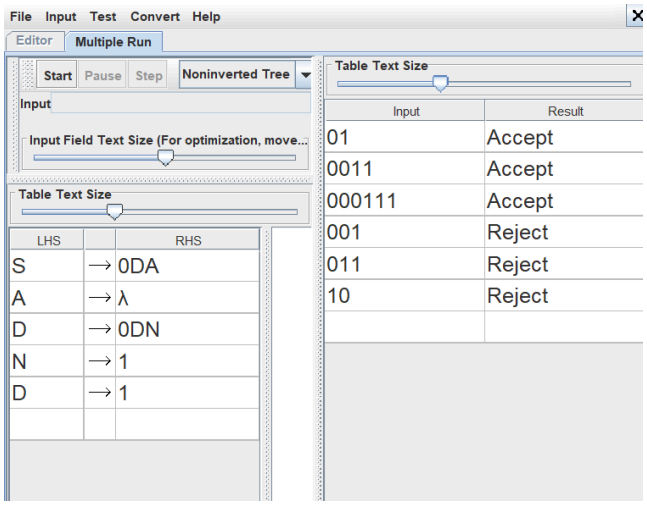
5. Save the PDA and select **Convert → Convert to Grammar**.
6. Complete the conversion and examine the generated grammar.

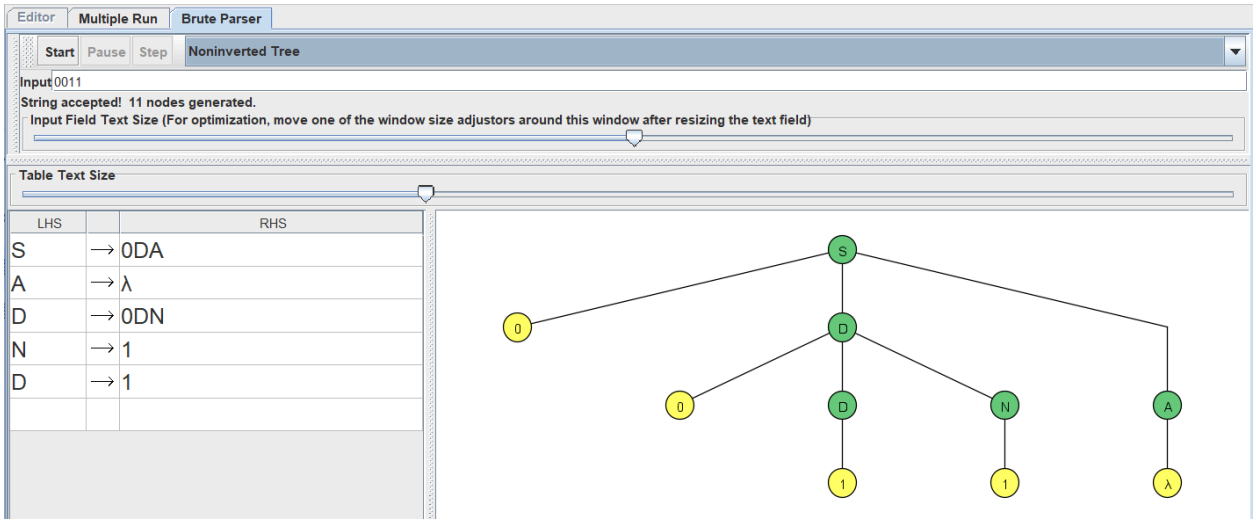










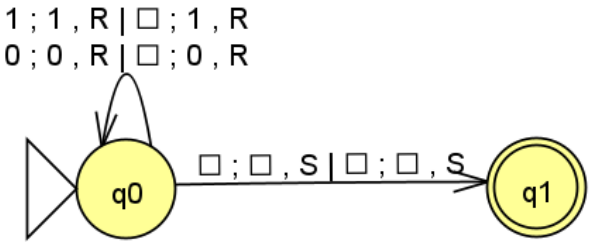


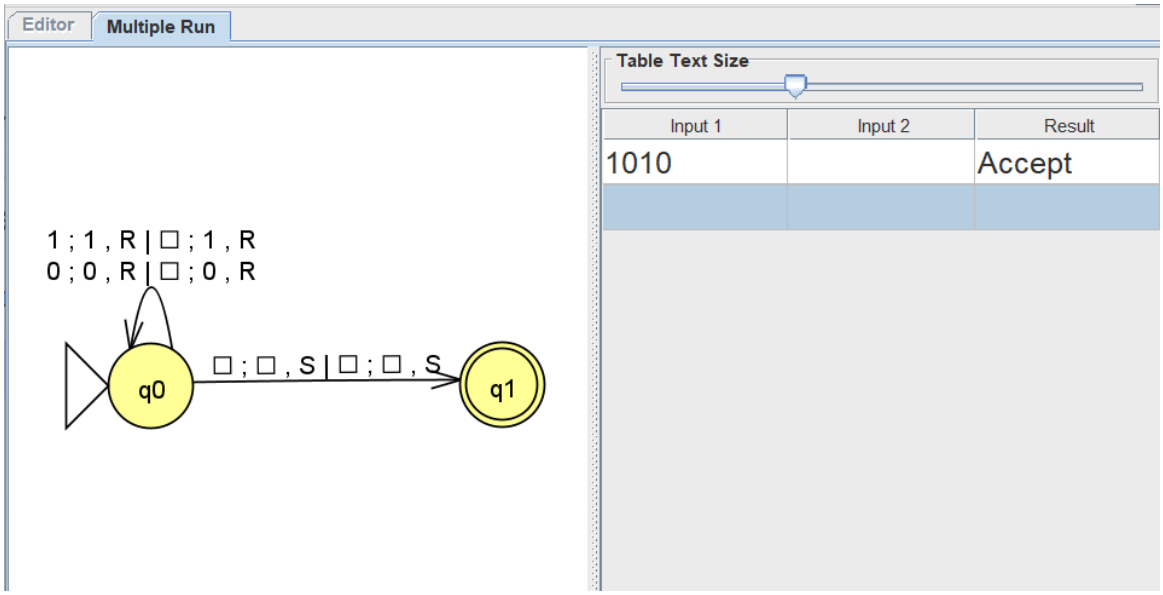
Result: An NPDA was constructed and tested for the language (L = {0^n1^n \mid n \geq 1}), and its conversion to a CFG was performed using JFLAP 7.1.

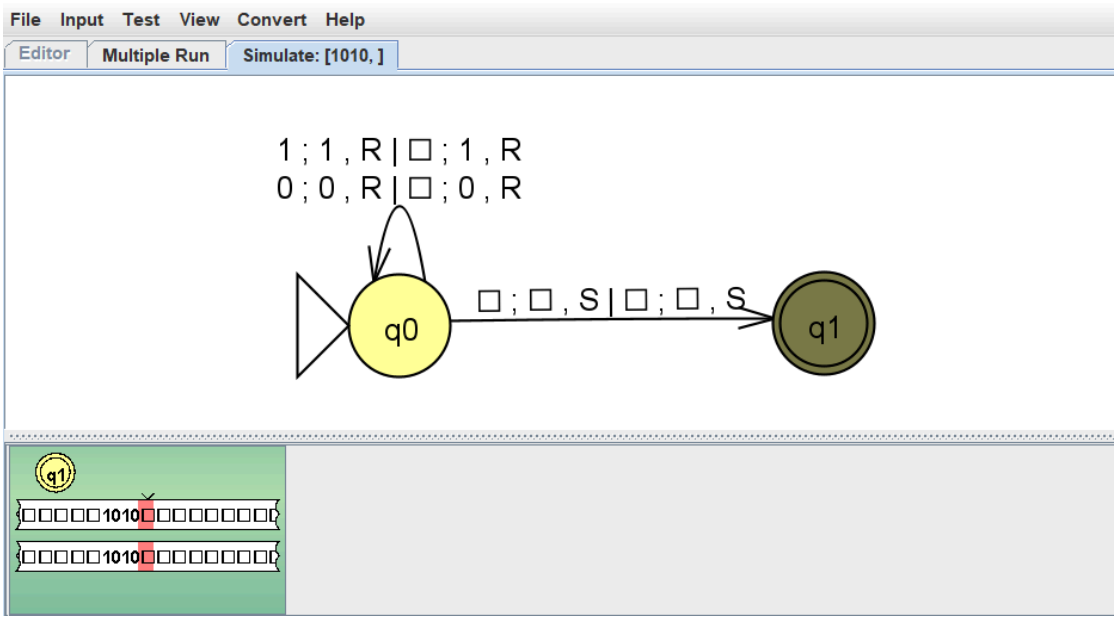
Question 10: Design a Two-Tape Turing Machine in JFLAP 7.1 to copy a binary input string from Tape 1 onto Tape 2.

Procedure:

1. Open JFLAP 7.1 and select **Multi-Tape Turing Machine**.
2. Create two states: (q0) and (q1). Mark (q0) as the initial state and (q1) as the final state.
3. Add a self-loop on (q0) for input `0`. The machine reads `0` on Tape 1, preserves it on Tape 1, writes `0` on Tape 2, and moves both tape heads to the right.
4. Add another self-loop on (q0) for input `1`. The machine reads `1` on Tape 1, preserves it on Tape 1, writes `1` on Tape 2, and moves both tape heads to the right.
5. Add a transition from (q0) to (q1) when both tape heads read blank symbols.
6. Test the machine using the input `1010` and observe the acceptance result.







Result: A Two-Tape Turing Machine was constructed in JFLAP 7.1 to process a binary input string. The machine accepted the test input `1010`.

Conclusion :